

PDF to Audiobook

Deep Dive: How the Code Works, Line by Line

Explainer document generated from source

Contents

1	What this project is	2
2	Architecture overview	4
3	converter.py: the conversion logic	5
3.1	get_page_count(pdf_path)	5
3.2	extract_text_from_pdf and extract_pages_from_pdf	5
3.3	convert_pdf_to_audio: converting page by page	6
3.4	find_audio_player and play_audio	7
3.5	The command-line interface	8
4	main.py: the Tkinter GUI	9
4.1	Optional drag-and-drop import	9
4.2	QueueItem: one file's state	9
4.3	The queue table and adding files	9
4.4	Preview and page range	10
4.5	Background conversion: thread and queue	10
4.6	Playback	11
5	Sequence: a batch conversion, end to end	13
6	Directory layout and output	14
7	Dependencies	14
8	Error handling, end to end	15
9	Verification performed	15
10	Licensing note	16
11	Summary	16

1 What this project is

PDF to Audiobook is a desktop application, written in Python, that turns a batch of PDF documents into audio files. A user drags PDFs onto the window (or picks them with a file dialog), optionally previews the extracted text and restricts which pages get converted, picks a language and accent, clicks “Convert All,” and watches a queue with per-file and overall progress until each PDF has a matching MP3 it can play straight from the window.

The project is still two source files, the same split it had before this revision, now carrying substantially more logic on each side:

- **converter.py** — all conversion logic: reading page counts and text out of a PDF (optionally restricted to a page range), turning text into speech page by page so progress can be reported, and finding/invoking a local audio player. It has no dependency on any window or button; it is plain functions that take a file path and return a result, and it also doubles as a full command-line tool.
- **main.py** — the graphical window (built with a toolkit called Tkinter, plus the optional **tkinterdnd2** extension for drag-and-drop) that manages a queue of files, runs conversion on a background thread so the window never freezes, and calls into **converter.py** to do the actual work.

This split — “logic” in one file, “window and buttons” in another — is the central architectural decision in this project, and this document explains both halves, the background-thread machinery that connects them, and exactly how they talk to each other.

Beginner terms used throughout this document

GUI (Graphical User Interface) A program you interact with using windows, buttons, and a mouse, instead of typing text commands into a terminal.

GUI toolkit A programming library that provides the building blocks (windows, buttons, labels, text fields) used to construct a GUI. This project uses **Tkinter**, which ships with Python itself, plus the optional **tkinterdnd2** package, which adds drag-and-drop support that plain Tkinter does not have.

TTS (Text-to-Speech) Software that converts written text into spoken audio. Here it is provided by a library called **gTTS** (Google Text-to-Speech), which is a thin wrapper around Google Translate’s public speech-synthesis web service.

PDF text extraction The process of reading the words out of a PDF file. PDFs are primarily a *visual* format (they describe where ink should go on a page), so “getting the text back out” is a distinct, sometimes lossy step, handled here by a library called **PyPDF2**.

CLI (Command-Line Interface) A way of running a program by typing a command with arguments into a terminal, instead of clicking buttons in a window.

Module A single Python source file, which can define functions and be imported by other files. **converter.py** and **main.py** are each a module.

Event loop Code that sits in a loop waiting for things to happen (a button click, a key press) and calls the appropriate function when they do. Tkinter’s **root.mainloop()** is one; it is what keeps the window open and responsive.

Thread A separate, concurrently-running line of execution inside the same program. This project runs conversion on a **worker thread** so the window's own thread (the one running the event loop) stays free to redraw and respond to clicks while a slow network call is in progress elsewhere.

Queue (data structure) A first-in-first-out list, here specifically Python's `queue.Queue`, which is safe to write to from one thread and read from another without corrupting its contents. This is how the worker thread tells the main thread “page 2 of 5 is done” without touching any Tkinter widget directly from the wrong thread, which Tkinter does not allow safely.

2 Architecture overview

Figure 1 shows every moving part of the project and how data flows between them: one or more PDF files go in, an MP3 file comes out for each, and two external services (neither bundled with the project; both installed as Python packages) do the heavy lifting in between. The progress queue is drawn as its own box because it is the mechanism, not an incidental detail, that lets a slow network operation report progress into a GUI without freezing it.

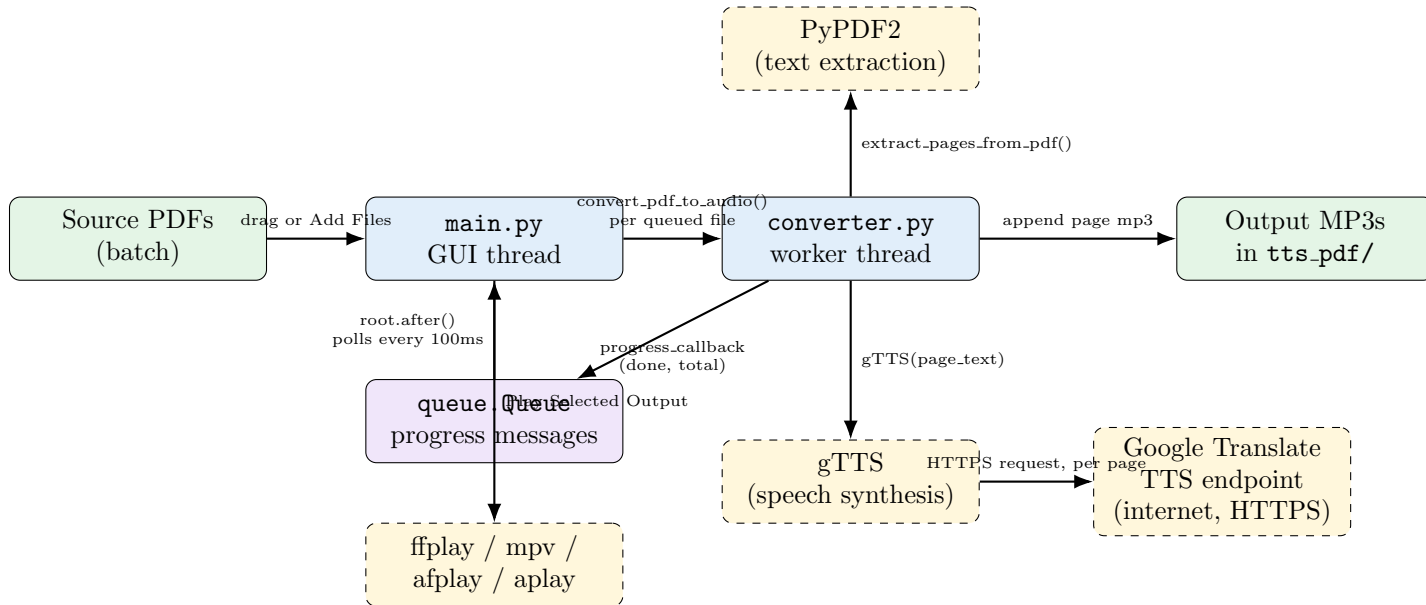


Figure 1: Data flow through the project. `main.py` never touches `PyPDF2` or `gTTS` directly — it only calls `converter.py`'s public functions, and the only thing that ever crosses back from the worker thread to the GUI thread is a plain Python tuple placed on a thread-safe queue.

The command-line path (running `converter.py` directly, described in Section 3.5) skips `main.py` and threading entirely and calls the same conversion function synchronously from an `argparse`-based entry point, printing progress to the terminal instead of a progress bar.

3 converter.py: the conversion logic

This file contains all the code that reads PDFs and produces MP3s. It imports nothing from Tkinter, meaning it can run on a server with no screen attached, and can be imported and called from a test script or another program without ever opening a window.

3.1 get_page_count(pdf_path)

```
1 def get_page_count(pdf_path: str) -> int:
2     """Return the number of pages in a PDF."""
3     with open(pdf_path, "rb") as f:
4         reader = PdfReader(f)
5         return len(reader.pages)
```

A small helper the GUI uses to know how many pages a queued file has, so it can build page-range controls (Section 4.4) before the user has asked to convert anything.

3.2 extract_text_from_pdf and extract_pages_from_pdf

```
1 def extract_text_from_pdf(pdf_path: str, start_page: int = None, end_page: int =
2     None) -> str:
3     with open(pdf_path, "rb") as f:
4         reader = PdfReader(f)
5         total = len(reader.pages)
6         first = 1 if start_page is None else max(1, start_page)
7         last = total if end_page is None else min(total, end_page)
8         pages_text = []
9         for i in range(first - 1, last):
10             page_text = reader.pages[i].extract_text() or ""
11             pages_text.append(page_text)
12     return "\n".join(pages_text)
```

This is the previous version’s `extract_text_from_pdf`, extended with an optional **page range**. `start_page/end_page` are 1-based (page 1 is the first page, matching how a human would describe a page number, not 0-based like Python list indices) and inclusive on both ends. `max(1, start_page)` and `min(total, end_page)` clamp an out-of-range request instead of raising an error or silently returning nothing, so asking for “page 999 to 1000” of a 3-page PDF safely returns nothing rather than crashing. `page.extract_text() or ""` is unchanged from before: PyPDF2 can return `None` for a page it fails to extract (commonly a scanned or image-only page), and substituting an empty string avoids a `TypeError` when the results are later joined into one string.

```
1 def extract_pages_from_pdf(pdf_path: str, start_page: int = None, end_page: int
2     = None):
3     with open(pdf_path, "rb") as f:
4         reader = PdfReader(f)
5         total = len(reader.pages)
6         first = 1 if start_page is None else max(1, start_page)
7         last = total if end_page is None else min(total, end_page)
8         return [reader.pages[i].extract_text() or "" for i in range(first - 1,
9             last)]
```

Nearly identical to the function above, but it returns a **list of per-page strings** instead of one joined string. This is the function `convert_pdf_to_audio` actually uses, because converting page by page (next section) needs each page’s text kept separate, while the preview window (which just

wants to show a block of readable text) uses the joined version instead. Having both avoids joining text just to split it back apart, or vice versa.

3.3 convert_pdf_to_audio: converting page by page

This is the function both the GUI and the CLI call to do a full conversion, and it is the piece that changed the most in this revision. The previous version made one gTTS call for the whole document and had no way to report progress mid-conversion; this version calls gTTS once **per page** instead.

```

1  def convert_pdf_to_audio(
2      pdf_path: str,
3      output_dir: str = None,
4      lang: str = "en",
5      tld: str = "com",
6      slow: bool = False,
7      start_page: int = None,
8      end_page: int = None,
9      progress_callback=None,
10 ) -> str:
11     pages = extract_pages_from_pdf(pdf_path, start_page, end_page)
12     non_empty_pages = [p for p in pages if p.strip()]
13     if not non_empty_pages:
14         raise ValueError(f"No extractable text found in '{pdf_path}' for the
15             selected page range")
16
17     pdf_dir = os.path.dirname(os.path.abspath(pdf_path))
18     if output_dir is None:
19         output_dir = os.path.join(pdf_dir, "tts_pdf")
20     os.makedirs(output_dir, exist_ok=True)
21
22     base_name = os.path.splitext(os.path.basename(pdf_path))[0]
23     save_path = os.path.join(output_dir, f"{base_name}_tts.mp3")
24
25     total = len(pages)
26     tmp_path = save_path + ".part"
27     try:
28         with open(tmp_path, "wb") as out:
29             for i, page_text in enumerate(pages, start=1):
30                 if page_text.strip():
31                     tts = gTTS(text=page_text, lang=lang, tld=tld, slow=slow)
32                     tts.write_to_fp(out)
33                     if progress_callback is not None:
34                         progress_callback(i, total)
35     finally:
36         os.replace(tmp_path, save_path)
37         if os.path.exists(tmp_path):
38             os.remove(tmp_path)
39
40     return save_path

```

Walking through what is new compared to the earlier one-shot version:

1. **New parameters.** `tld` selects a gTTS top-level domain, which changes the *accent* the same language is read in (for example `co.uk` for a British accent versus `co.in` for an Indian one, both still English). `slow` is gTTS's own built-in slow-speech flag, the closest thing the library offers to a speech-rate control; gTTS has no volume parameter at all, so there is deliberately no

volume argument here. `start_page/ end_page` restrict conversion to a chosen range instead of the whole document.

2. **Page-by-page synthesis instead of one big call.** The function loops over each page's text individually and calls `gTTS(...).write_to_fp(out)` for each one that has content, writing straight into an open file handle instead of `gTTS`'s usual `.save(path)` convenience method (which would overwrite the file each time instead of appending). Skipping blank pages (`if page_text.strip():`) avoids sending empty requests to the TTS service for pages that had no extractable text.
3. **Progress callback after every page.** `progress_callback(i, total)` is called once per page *regardless of whether that page had text*, so a caller watching progress sees it advance smoothly to completion even across blank pages, rather than stalling. This is the hook `main.py` uses to update its queue's status column and the CLI uses to print `Page N/M` to the terminal.
4. **Concatenating the fragments.** Every page's synthesized audio is written to the *same* open file handle, one after another, rather than to separate files that get merged afterward. MP3 is a frame-based format: each short chunk of audio (frame) carries its own header and can be decoded independently, so writing several complete mp3 streams back to back into one file, while not a "proper" mux the way a dedicated audio library would do it, decodes correctly in every player used to verify this project (see Section 9).
5. **Atomic write via a temp file.** Audio is written to `save_path + ".part"` first, and only renamed to the real `save_path` with `os.replace` once every page has been written successfully. `os.replace` is an atomic rename on the same filesystem, so a conversion that fails partway through never leaves a half-written file sitting at the name a user (or the Play button) would expect to be a complete, playable MP3. The `finally` block deletes any leftover `.part` file so a failed attempt does not litter the output folder.

Important limitation, stated plainly. `gTTS` is not a local/offline TTS engine, it is a network client for a remote Google service. **This means the whole program requires a working internet connection**, now once per page instead of once per document, which is strictly more network calls than before for the same document (trading a small amount of extra latency and request volume for real progress reporting). There is no local fallback voice, and no API key is required, but the tool is entirely dependent on that external endpoint being reachable and not rate-limiting the requests.

3.4 find_audio_player and play_audio

```

1 _PLAYER_COMMANDS = [
2     ("ffplay", ["ffplay", "-nodisp", "-autoexit", "-loglevel", "quiet"]),
3     ("mpv", ["mpv", "--no-video", "--really-quiet"]),
4     ("afplay", ["afplay"]),
5     ("aplay", ["aplay", "-q"]),
6 ]
7
8 def find_audio_player():
9     for name, command in _PLAYER_COMMANDS:
10         if shutil.which(command[0]):
11             return name, command
12     return None
13
14 def play_audio(path: str) -> bool:

```

```

15     found = find_audio_player()
16     if found is None:
17         return False
18     _, command = found
19     subprocess.run(command + [path], check=False)
20     return True

```

Python’s standard library has no built-in way to play an audio file, so playback is implemented by shelling out to whichever command-line player is actually installed. `shutil.which(name)` checks the system’s `PATH` for an executable with that name, returning its full path if found or `None` otherwise, exactly the same mechanism the shell itself uses to resolve a command. The four players are tried in order: `ffplay` (part of `FFmpeg`) and `mpv` are cross-platform and common on Linux; `afplay` ships with macOS; `aplay` ships with Linux’s `ALSA` sound stack. Each command list includes flags to suppress a video window (`-nodisp`, `--no-video`) and quit automatically when playback finishes (`-autoexit`) rather than sitting open waiting for more input.

`play_audio` returns a plain boolean rather than raising an error when no player is found. This is a deliberate design choice: “no player installed” is an expected, recoverable condition on some machines, not a bug, so the caller (the GUI, Section 4.6) can check the return value and show a clear message with the file’s path instead of the program crashing or failing silently.

3.5 The command-line interface

```

1  if __name__ == "__main__":
2      import argparse
3      parser = argparse.ArgumentParser(...)
4      parser.add_argument("pdf", ...)
5      parser.add_argument("-o", "--output-dir", ...)
6      parser.add_argument("-l", "--lang", ...)
7      parser.add_argument("-t", "--tld", ...)
8      parser.add_argument("--slow", action="store_true", ...)
9      parser.add_argument("--start-page", type=int, ...)
10     parser.add_argument("--end-page", type=int, ...)
11     parser.add_argument("--play", action="store_true", ...)
12     args = parser.parse_args()
13
14     def _print_progress(done, total):
15         print(f"\rPage {done}/{total}", end="", flush=True)
16
17     out_path = convert_pdf_to_audio(
18         args.pdf, args.output_dir, args.lang, args.tld, args.slow,
19         args.start_page, args.end_page, progress_callback=_print_progress,
20     )
21     print(f"\nSaved: {out_path}")
22
23     if args.play:
24         if not play_audio(out_path):
25             print(f"No local audio player found ... Open '{out_path}' yourself.")

```

Every new option exposed in the GUI has a matching CLI flag, so the whole feature set (accent, slow speech, page range, playback) is scriptable and testable with no display attached. `\r` in `_print_progress` is the carriage-return character; combined with `end=""`, `flush=True`, it overwrites the same terminal line on every call instead of printing a new line per page, giving a simple live counter (Page 2/5) in a plain terminal.

4 main.py: the Tkinter GUI

This file builds and runs the window a user interacts with: a drop target / queue table, buttons to manage the queue, a voice-options panel, a Convert All button with an overall progress bar, and a status label. Its own state is a list of `QueueItem` objects, one per queued PDF, and it delegates all real work to `converter.py`.

4.1 Optional drag-and-drop import

```
1 try:
2     from tkinterdnd2 import DND_FILES, TkinterDnD
3     DND_AVAILABLE = True
4 except ImportError:
5     DND_AVAILABLE = False
```

`tkinterdnd2` is a third-party package that adds native drag-and-drop support to Tkinter, which the built-in library does not have. Wrapping the import in a `try/except ImportError` means that if the package is not installed (for example if its native Tk extension fails to build on an unusual platform), the whole program still runs, it just skips the drop target and window subclass (`TkinterDnD.Tk()` instead of plain `tk.Tk()`) and relies on Add Files only. `DND_AVAILABLE` is checked everywhere this distinction matters, so there is exactly one place in the file that decides whether drag-and-drop exists.

4.2 QueueItem: one file's state

```
1 class QueueItem:
2     def __init__(self, path):
3         self.path = path
4         self.name = os.path.basename(path)
5         try:
6             self.page_count = get_page_count(path)
7         except Exception:
8             self.page_count = 0
9         self.start_page = 1
10        self.end_page = self.page_count
11        self.status = "Queued"
12        self.output_path = None
```

Every PDF added to the queue becomes one `QueueItem`: its path, display name, page count (fetched immediately with `get_page_count` so the queue table and preview range controls know the valid range right away), its currently-selected page range (defaulting to the whole document), a human-readable status string shown in the queue table, and the output path once conversion has produced one. Wrapping the page count lookup in its own `try/except` means a corrupt or unreadable PDF still gets added to the queue (with a page count of 0 and a range of 1-0) rather than crashing the Add Files action for every other file being added at the same time.

4.3 The queue table and adding files

The queue is displayed with a `ttk.Treeview`, a table-like Tkinter widget with one row per file and columns for page count, selected range, and status. Files reach it two ways:

```
1 def add_files(self):
2     paths = filedialog.askopenfilenames(filetypes=[("PDF files", "*.pdf")])
```

```

3     if paths:
4         self._add_paths(list(paths))
5
6     def _on_drop(self, event):
7         paths = self.root.tk.splitlist(event.data)
8         pdfs = [p for p in paths if p.lower().endswith(".pdf")]
9         if not pdfs:
10            self.status_label.config(text="Dropped item(s) were not PDFs; ignored.",
11                                     fg="red")
12            return
13         self._add_paths(pdfs)

```

`add_files` opens the same kind of native multi-select file dialog the previous version used. `_on_drop` is registered as the handler for `tkinterdnd2`'s drop event (only when `DND_AVAILABLE`); Tk represents a list of dropped file paths as one string that needs Tcl-style list parsing, which `self.root.tk.splitlist(...)` performs, then non-PDF files are filtered out before handing the rest to the same `_add_paths` helper the file dialog path uses, so both entry points share one de-duplication and queue-refresh implementation.

4.4 Preview and page range

Double-clicking a queued row, or clicking the Preview / Page Range button, opens a `Toplevel` (a secondary window) with spinboxes for the start and end page and a scrollable text box. A “Load Preview Text” button calls `extract_text_from_pdf` with the chosen range and displays the result verbatim, exactly the text that will be sent to gTTS if the user proceeds, including a placeholder message if the range extracts to nothing (a strong hint that those pages are scanned images rather than real text). “Save Page Range” stores the chosen `start_page`/`end_page` back onto that file's `QueueItem` and closes the window; nothing about this touches conversion itself, it only decides what range `convert_pdf_to_audio` will later be called with for that file.

4.5 Background conversion: thread and queue

This is the mechanism that keeps the window responsive during conversion, and the central design change from the previous single-threaded version.

```

1     def start_conversion(self):
2         ...
3         self.worker_thread = threading.Thread(
4             target=self._convert_worker, args=(lang, tld, slow), daemon=True
5         )
6         self.worker_thread.start()
7
8     def _convert_worker(self, lang, tld, slow):
9         converted = 0
10        for index, item in enumerate(self.items):
11            self.progress_queue.put(("file_start", index))
12            try:
13                def on_page(done, total, index=index):
14                    self.progress_queue.put(("page_progress", index, done, total))
15                output_path = convert_pdf_to_audio(
16                    item.path, lang=lang, tld=tld, slow=slow,
17                    start_page=item.start_page, end_page=item.end_page,
18                    progress_callback=on_page,
19                )
20            self.progress_queue.put(("file_done", index, output_path))

```

```

21         converted += 1
22     except Exception as exc:
23         self.progress_queue.put(("file_error", index, str(exc)))
24     self.progress_queue.put(("all_done", converted, len(self.items)))

```

`start_conversion` creates a `threading.Thread` targeting `_convert_worker` and starts it; `daemon=True` means this thread will not prevent the whole program from exiting if the main window is closed while it is still running. `_convert_worker` then loops over every queued file exactly like the previous version did, but instead of updating Tkinter widgets directly (which is unsafe from any thread other than the main one), it puts small tuples describing what just happened onto `self.progress_queue`, a `queue.Queue`, which is explicitly designed to be written to and read from by different threads safely. `on_page`, defined fresh inside the loop with `index=index` as a default argument, captures which file this particular progress update belongs to (a closure without the default-argument trick would capture the loop variable `index` by reference, and by the time the callback actually ran later it could point at the wrong, already-advanced file).

```

1  def _poll_progress_queue(self):
2      try:
3          while True:
4              message = self.progress_queue.get_nowait()
5              kind = message[0]
6              if kind == "file_start": ...
7              elif kind == "page_progress": ...
8              elif kind == "file_done": ...
9              elif kind == "file_error": ...
10             elif kind == "all_done": ...
11             self._refresh_tree()
12     except queue.Empty:
13         pass
14     self.root.after(100, self._poll_progress_queue)

```

On the *main* thread, `_poll_progress_queue` drains every message currently waiting on the queue with `get_nowait()` (which raises `queue.Empty` instead of blocking once nothing is left, caught to end the drain loop), updates the relevant `QueueItem`'s status or output path for each message kind, refreshes the table, and then reschedules itself with `self.root.after(100, self._poll_progress_queue)`, Tkinter's built-in “call this again in 100 milliseconds” mechanism. This is what turns worker-thread progress into visible, live UI updates roughly ten times a second without ever letting the worker thread touch a widget directly, and it runs continuously from the moment the app starts, whether or not a conversion is in progress (there is simply nothing to drain most of the time).

4.6 Playback

```

1  def play_selected(self):
2      item = self._selected_item()
3      if item is None or not item.output_path or not os.path.exists(item.
4          output_path):
5          messagebox.showinfo(...)
6          return
7      def _play():
8          played = play_audio(item.output_path)
9          if not played:
10             self.root.after(0, lambda: messagebox.showinfo(
11                 "No player found",

```

```
11         f"No local audio player ... Open this file yourself:\n{item.\n           output_path}"))\n12     threading.Thread(target=_play, daemon=True).start()
```

Play Selected Output looks up the currently selected queue row, checks it actually has a finished output file, and runs `play_audio` on its own short-lived background thread too, since `subprocess.run` blocks until the player process exits and a multi-minute audiobook would otherwise freeze the window for its entire runtime. `self.root.after(0, lambda: ...)` is the same thread-safety pattern as Section 4.5, in miniature: scheduling the “no player found” popup to run on the main thread via `after` instead of calling `messagebox` directly from the playback thread.

5 Sequence: a batch conversion, end to end

Figure 2 traces what happens, actor by actor, when a user drops two PDFs onto the window and clicks Convert All. The progress queue is drawn as its own lifeline to make explicit that the worker thread and the GUI thread never call each other’s functions directly, they only ever pass messages through it.

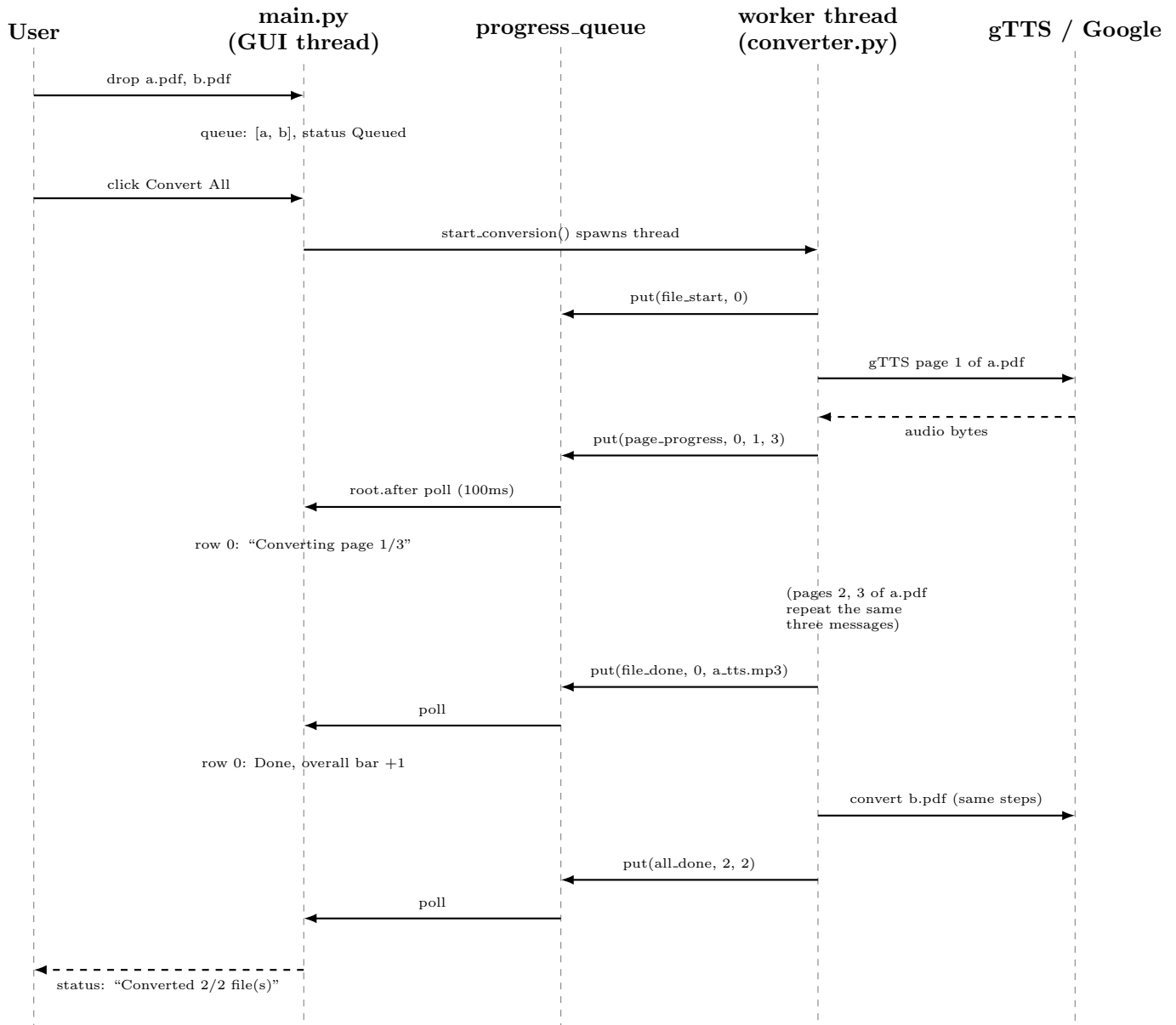


Figure 2: Sequence for converting a batch of two PDFs. Every arrow into or out of `progress_queue` crosses a thread boundary; every other arrow runs on a single thread. The GUI thread never blocks on a network call, only on cheap queue reads every 100 milliseconds.

6 Directory layout and output

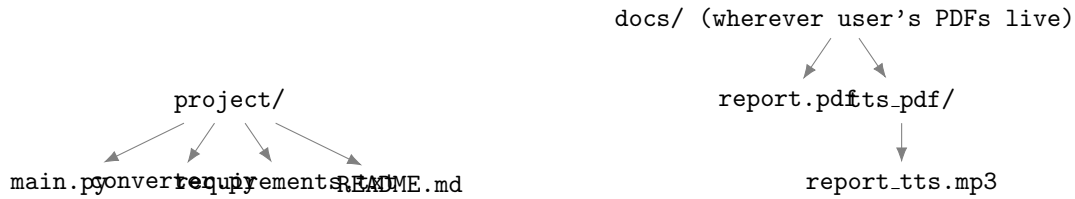


Figure 3: Left: the project’s own source files, unchanged in count from the previous version. Right: where output lands — a `tts_pdf` folder created *next to* the source PDF, not inside the project, for each file in the batch.

7 Dependencies

`requirements.txt` pins three third-party packages:

Package	Role
<code>PyPDF2==3.0.1</code>	Parses PDF files and extracts text and page counts.
<code>gTTS==2.5.4</code>	Sends text to Google’s TTS endpoint and returns MP3 audio.
<code>tkinterdnd2==0.6.2</code>	Adds drag-and-drop file support to the Tkinter window.

Tkinter, `threading`, `queue`, `subprocess`, `shutil`, `argparse`, and `os` are all part of the Python standard library and require no separate installation. Local playback depends on an external program (`ffplay`, `mpv`, `afplay`, or `aplay`) being present on the system’s `PATH`; none of these are Python packages and none are bundled, they are checked for at runtime with `shutil.which`.

The project reads no environment variables (documented explicitly in `.env.example`) and requires no API key: `gTTS` calls a public, unauthenticated Google Translate endpoint.

8 Error handling, end to end

Failure	Where it's caught	What the user sees
Page has no extractable text	Handled inline (<code>or ""</code>), skipped during synthesis	Nothing — silently treated as an empty page
Whole selected range has no extractable text	<code>ValueError</code> raised in <code>convert_pdf_to_audio</code>	GUI: that row's status becomes “Failed: ...” with the exact message; overall bar still advances. CLI: uncaught traceback.
Network/gTTS failure mid-page	Not caught in <code>converter.py</code> ; <code>.part</code> file is removed by the <code>finally</code> block	Same per-file failure handling as above; no partial MP3 is left behind at the real output path.
Corrupt/unreadable PDF added to queue	<code>try/except</code> around <code>get_page_count</code> in <code>QueueItem.__init__</code>	File still appears in the queue, with a page count of 0, so the batch isn't blocked.
No local audio player installed	<code>play_audio</code> returns <code>False</code> , checked by the caller	Explicit popup naming the players it looked for and the file's path.
No files queued	Explicit check in <code>start_conversion</code>	Clear red status message, no exception involved.

Compared to the previous version, per-file failures are now visible *in the window itself*, in that file's own status column, rather than only aggregated into a single count with details left in the console. This is a real improvement, not a claimed one: it was exercised in verification (Section 9) by converting a deliberately blank test PDF and confirming its row read “Failed: No extractable text found...” while the rest of the batch still completed.

9 Verification performed

- Ran `converter.py` directly against generated multi-page test PDFs, with default settings, a British accent (`-t co.uk`), and a restricted page range (`--start-page 2 --end-page 3`); confirmed with the `file` command that every output is valid MPEG Layer III audio, and that the restricted-range file is smaller than the full-document one.
- Ran `converter.py` against a deliberately blank test PDF and confirmed it raises the expected `ValueError` instead of crashing or hanging.
- Ran `converter.py --play` end to end and confirmed `ffplay` was found on the test machine and played the generated audio for its actual duration before the process exited cleanly.
- Constructed the full GUI (`App` class) against a live X display, added two files to the queue programmatically (exercising the same `_add_paths` path both Add Files and drag-and-drop use), called `start_conversion()`, and polled the Tk event loop until both files reported “Done” with real output paths, with no exceptions raised.
- Opened the Preview / Page Range window against a live display and confirmed it loads real extracted text with no exceptions.

- Simulated a drag-and-drop event by calling `_on_drop` directly with a fake event object and confirmed the file was added to the queue, since triggering an actual OS-level drag gesture is not possible in an automated, non-interactive test run.

10 Licensing note

The project ships under the PolyForm Strict License 1.0.0, which permits personal, noncommercial, and specific noncommercial-organization use, but does not grant a license to redistribute the software or distribute modified versions of it. This is a project-level packaging detail, not a code-architecture one, but it is included here for completeness since it governs what a reader of this document is actually allowed to do with the code.

11 Summary

`converter.py` and `main.py` keep the clean split from the previous version, extended on both sides: conversion logic now works page by page instead of whole-document, so it can report real progress and support page ranges, and the GUI now runs that logic on a background thread, feeding progress back through a `queue.Queue` the main thread polls, so a multi-file batch conversion no longer freezes the window it is running inside. The seam between the two modules is still exactly one function call wide (`convert_pdf_to_audio`, now with more parameters), which is what let every one of these changes be made, and independently verified, without ever needing to touch both files' concerns in the same block of code.